



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

PROYECTO

"APLICACIÓN MÓVIL COLABORATIVA PARA MEJORAR LA EFECTIVIDAD EN LA ASISTENCIA DE MASCOTAS PERDIDAS CON PARTICIPACIÓN DE VOLUNTARIOS EN LA CIUDAD DE TACNA, 2026"

Curso: Construcción de software II

Docente: RICARDO EDUARDO VALCARCEL
ALVARADO

Integrantes:

<i>Chite Quispe Brian Danilo</i>	<i>(2021070015)</i>
<i>Anampa Pancca David Jordan</i>	<i>(2022074268)</i>

Tacna – Perú
2026

INDICE

1. Introducción.....	3
2. Objetivo del Plan.....	3
3. Alcance.....	4
4. Tipos de Mantenimiento.....	4
5. Fase 1: Proceso de Implementación y Recepción de Solicitudes	6
5.1 Recepción de Solicitudes de Modificación	6
5.2 Plantilla de Formulario de Solicitud de Modificación	6
5.3 Priorización de Solicitudes.....	7
5.4 Ejemplo de Matriz de Priorización Aplicada	8
6. Fase 2: Análisis de Modificación y Problemas	8
6.1 Diagnóstico del Problema	8
6.2 Definición del Tipo de Mantenimiento por Módulo	9
6.3 Plantilla de Informe de Diagnóstico	10
7. Fase 3: Implementación de la Modificación	10
7.1 Flujo de Trabajo Git	10
7.2 Estándares de Desarrollo Obligatorios	10
7.3 Code Review Obligatorio	11
7.4 Documentación de la Modificación	11
8. Fase 4: Aceptación y Revisión del Mantenimiento	12
8.1 Validación por el Área Solicitante	12
8.2 Checklist de Revisión Final Antes del Despliegue.....	12
9. Fase 5: Migración y Despliegue de la Modificación.....	13
9.1 Planificación de Migración	13
9.2 Respaldo y Recuperación.....	13
9.3 Smoke Tests Post-Despliegue	14
10. Fase 6: Retiro de Componentes Obsoletos	14
10.1 Componentes Ya Retirados (Laboratorio 01)	14
10.2 Proceso de Retiro Futuro	15
11. Métricas y Monitoreo del Mantenimiento.....	15
12. Roles y Responsabilidades	16
13. Referencias.....	17

1. Introducción

El presente documento establece el Plan de Mantenimiento de Software para el sistema **SOS Mascota**, una aplicación móvil colaborativa desarrollada en Flutter con backend en Firebase (Authentication, Cloud Firestore, Cloud Storage, Cloud Messaging) e inferencia local de visión artificial mediante TensorFlow Lite.

El sistema se encuentra en fase de construcción activa dentro del repositorio `David-Anampa/CONSTRUCCION_SOFTWARE_II_ANAMPA_CHITE`, rama `UNIDAD-I`. Las características técnicas actuales del sistema son: arquitectura **MVVM** (Model-View-ViewModel) con gestor de estados `Provider`, base de datos NoSQL en Cloud Firestore, autenticación con Firebase Authentication y verificación de identidad contra API RENIEC, integración con TensorFlow Lite para clasificación de mascotas y extracción de embeddings, generación de afiches en PDF, despacho de notificaciones push mediante FCM HTTP v1, y sistema de mapa interactivo con OpenStreetMap.

Este plan se estructura conforme a la norma internacional **ISO/IEC 14764:2006**, que define los procesos del ciclo de vida del software relacionados con el mantenimiento, estableciendo un marco sistemático para gestionar todas las actividades de modificación, corrección y mejora del sistema SOS Mascota.

2. Objetivo del Plan

El objetivo principal de este plan es establecer un proceso estructurado y controlado para el mantenimiento del sistema SOS Mascota, asegurando que todas las modificaciones se realicen de manera documentada, trazable y conforme a los estándares de calidad del curso de Construcción de Software II de la Universidad Privada de Tacna.

El plan busca garantizar:

- La continuidad operativa del sistema de rescate de mascotas en la ciudad de Tacna.
- La minimización de riesgos de regresión en producción que puedan perjudicar a los voluntarios y dueños de mascotas activos.
- La integridad de los datos históricos de reportes, avistamientos y documentos de identidad validados por RENIEC.
- La compatibilidad continua del sistema con las últimas versiones de Flutter SDK, Firebase SDK y TensorFlow Lite.

3. Alcance

Este plan aplica a todas las actividades de mantenimiento del sistema SOS Mascota, incluyendo:

- Corrección de errores en funcionalidades existentes (módulo de autenticación, registro de avistamientos, algoritmo de similitud coseno, despacho de notificaciones FCM).
- Adaptaciones a nuevas versiones del Flutter SDK, cambios en las APIs de Firebase Firestore, Cloud Messaging HTTP v1, y la API RENIEC.
- Mejoras de rendimiento en el algoritmo de cotejo biométrico, consultas Firestore y optimización del APK.
- Actualizaciones de seguridad para proteger los datos personales de voluntarios y las credenciales de la API RENIEC.
- Migraciones entre versiones del sistema y gestión de la rama UNIDAD-I del repositorio.
- Retiro de componentes obsoletos (archivos duplicados `pantalla_registro copy.dart`, `registro_vm copy.dart` ya eliminados en Laboratorio 01).

Los destinatarios del sistema y, por tanto, los afectados por el mantenimiento son:

- Dueños de mascotas extraviadas en la ciudad de Tacna.
- Voluntarios ciudadanos que registran avistamientos en la vía pública.
- Administradores del sistema que moderan usuarios y reportes.
- El equipo docente de la Universidad Privada de Tacna que evalúa el sistema.

4. Tipos de Mantenimiento

El sistema SOS Mascota requiere cuatro tipos de mantenimiento según la naturaleza de las modificaciones:

Tipo	Descripción	Ejemplo en SOS Mascota
Correctivo	Corrección de errores y defectos encontrados en producción	Corregir bug en <code>AuthWrapper</code> que no detecta en tiempo real cuando un administrador bloquea una cuenta activa. Corregir error en el algoritmo de similitud coseno que genera coincidencias falsas positivas con umbrales incorrectos.

Tipo	Descripción	Ejemplo en SOS Mascota
Adaptativo	Adaptación a cambios en el entorno tecnológico	Actualizar Flutter SDK de 3.29 a versión posterior. Adaptar <code>NotificacionServicio</code> cuando FCM deprecie el protocolo HTTP v1 o cambie la estructura de payloads. Actualizar <code>tflite_flutter</code> a nueva versión con API modificada.
Perfectivo	Mejoras de rendimiento, usabilidad y funcionalidad	Implementar <code>compute()</code> de Flutter para ejecutar el algoritmo de similitud coseno en un isolate separado. Mejorar la interfaz del wizard de 3 pasos para reporte de mascotas. Agregar filtro por raza en el feed principal de reportes.
Preventivo	Acciones proactivas para prevenir problemas futuros	Refactorizar los ViewModels de mayor longitud (<code>admin_vm.dart</code>) para mejorar su mantenibilidad. Implementar logging estructurado en todos los servicios para facilitar diagnóstico. Actualizar periódicamente las dependencias en <code>pubspec.yaml</code> .

Fuente: Elaboración propia del equipo de trabajo.

5. Fase 1: Proceso de Implementación y Recepción de Solicitudes

La fase de proceso de implementación establece el mecanismo para recibir, registrar, priorizar y documentar todas las solicitudes de modificación del sistema SOS Mascota.

5.1 Recepción de Solicitudes de Modificación

El sistema SOS Mascota recibe solicitudes de modificación a través de los siguientes canales:

- Sistema de gestión de **Issues en GitHub** (repositorio: **David-Anampa/CONSTRUCCION_SOFTWARE_II_ANAMPA_CHITE**).
- Reporte directo de usuarios de la comunidad de Tacna a través del sistema de comentarios interno de la app.
- Feedback del docente evaluador durante las sesiones de revisión del laboratorio.

Cuando se recibe una solicitud, el equipo de soporte debe registrar inmediatamente la información en el sistema de gestión, asignando un identificador único con el formato **MANT-YYYYMMDD-XXX** (por ejemplo, **MANT-20261015-001**). Este identificador permite la trazabilidad completa de la solicitud a través de todo el proceso de mantenimiento.

5.2 Plantilla de Formulario de Solicitud de Modificación

Campo	Detalle
ID de Solicitud	MANT-YYYYMMDD-XXX
Fecha de Recepción	DD/MM/AAAA HH:MM
Solicitante	Nombre y rol (Voluntario / Administrador / Docente)
Tipo de Solicitud	Bug / Mejora / Cambio / Consulta
Módulo Afectado	Autenticación / Reportes / Avistamientos / Notificaciones / Administración / IA / Mapa / Chat / PDF
Descripción del Problema	Descripción detallada del error o necesidad
Pasos para Reproducir	Secuencia de acciones que reproducen el problema
Prioridad Estimada	P0 Crítica / P1 Alta / P2 Media / P3 Baja
Evidencia Adjunta	Capturas de pantalla, logs, grabaciones de pantalla

5.3 Priorización de Solicitudes

La priorización de solicitudes en SOS Mascota es crítica porque el sistema atiende rescates de mascotas en tiempo real. Una solicitud que afecte el registro de avistamientos o el despacho de notificaciones tiene mayor impacto que una mejora cosmética en la interfaz.

Nivel	Criterio	Tiempo Máximo de Respuesta	Ejemplo en SOS Mascota
P0 Crítica	Bloquea completamente la operación	24 horas	Fallo en Firebase Authentication que impide el login a todos los usuarios. Fallo en Firestore que impide guardar reportes de mascotas perdidas.
P1 Alta	Afecta funcionalidad importante sin bloquear	5 días hábiles	Error en el algoritmo de similitud coseno que no detecta coincidencias válidas. Fallo en el despacho de notificaciones FCM al dueño de la mascota.
P2 Media	Mejora la experiencia sin ser urgente	15 días hábiles	Agregar filtros adicionales en el feed de reportes. Optimizar la velocidad de carga de imágenes en el mapa interactivo.
P3 Baja	Mejora cosmética u opcional	Backlog	Cambiar la paleta de colores del Dashboard estadístico. Agregar animación de transición entre pantallas.

Fuente: Elaboración propia del equipo de trabajo.

5.4 Ejemplo de Matriz de Priorización Aplicada

ID Solicitud	Descripción	Impacto	Urgencia	Prioridad
MANT-20261015-001	Los avistamientos no se están guardando en Firestore debido a un cambio en las reglas de seguridad	Alto — Bloquea el registro de toda la comunidad	Alta — Sistema crítico inoperativo	P0 Crítica
MANT-20261015-002	Las notificaciones push no llegan cuando el dueño tiene la app cerrada en segundo plano	Alto — Dueño no se entera de coincidencias	Media	P1 Alta
MANT-20261015-003	El mapa interactivo tarda más de 5 segundos en cargar con más de 50 marcadores	Medio — Degradación de experiencia	Baja — No es urgente	P2 Media

Fuente: Elaboración propia del equipo de trabajo.

6. Fase 2: Análisis de Modificación y Problemas

La fase de análisis determina la solución técnica correcta antes de implementar cambios. En SOS Mascota, un análisis incorrecto puede llevar a soluciones que rompan la integración entre módulos (por ejemplo, modificar `NotificacionServicio` sin considerar que es usado por `ReporteMascotaVM` y `AvistamientoVM`) o afecten la integridad de los vectores de embeddings almacenados.

6.1 Diagnóstico del Problema

El diagnóstico comienza con la reproducción del problema en un ambiente de desarrollo que replica la configuración de producción. Para SOS Mascota, esto implica:

- Acceso a un proyecto de Firebase separado para desarrollo.
- Emulador de Firebase Firestore con datos de prueba representativos.

- Dispositivo Android físico (o emulador AVD Android 13) con los modelos TFLite cargados.
- Cuenta de prueba para la API RENIEC con token Bearer de testing.

El analista recopila información detallada:

- Logs del aplicativo Flutter obtenidos mediante flutter logs o el panel de depuración de Android Studio.
- Logs de Firebase desde la consola de Firebase (Authentication, Firestore, Functions).
- Capturas de pantalla o grabaciones del comportamiento incorrecto.
- Pasos exactos para reproducir el problema.

6.2 Definición del Tipo de Mantenimiento por Módulo

Módulo	Mantenimiento Correctivo Ejemplo	Mantenimiento Adaptativo Ejemplo
auth_servicio.dart	Bug en signOut() que no limpia el estado local de Provider	Adaptar a nuevo método de Firebase Auth SDK
servicio_tflite.dart	Error en normalización de tensor que genera embeddings nulos	Actualizar API de tflite_flutter tras cambio de versión
notificacion_servicio.dart	Fallo en construcción del payload FCM HTTP v1	Adaptar a nueva versión del protocolo FCM
avistamiento_vm.dart	Bug en cálculo de distancia euclidiana con coordenadas negativas	Adaptar fórmula Haversine para mayor precisión
imagen_util.dart	Error en descarga temporal de foto de reporte para comparación	Migrar a nuevo endpoint de Firebase Storage

6.3 Plantilla de Informe de Diagnóstico

Campo	Detalle
ID de Solicitud	MANT-YYYYMMDD-XXX
Síntoma Observado	Descripción del comportamiento incorrecto reportado
Causa Raíz Identificada	Causa técnica subyacente (línea de código, configuración, dependencia)
Archivos Afectados	Lista de archivos lib/ y test/ relacionados
Tipo de Mantenimiento	Correctivo / Adaptativo / Perfectivo / Preventivo
Solución Propuesta	Descripción de la solución técnica
Esfuerzo Estimado	Horas de desarrollo + pruebas + documentación
Riesgos de Implementación	Posibles efectos laterales en otros módulos
Analista Responsable	Nombre del desarrollador asignado

7. Fase 3: Implementación de la Modificación

La fase de implementación es donde se aplican los cambios al código fuente del sistema SOS Mascota. Esta fase debe seguir estrictamente los estándares de desarrollo del proyecto.

7.1 Flujo de Trabajo Git

El desarrollador crea una rama desde UNIDAD-I usando el formato:

- fix/MANT-YYYYMMDD-XXX para correcciones de errores.
- feature/MANT-YYYYMMDD-XXX para nuevas funcionalidades o mejoras.
- refactor/MANT-YYYYMMDD-XXX para refactorizaciones sin cambio de comportamiento.

7.2 Estándares de Desarrollo Obligatorios

Durante la implementación, el desarrollador debe seguir los estándares auditados en el Laboratorio 01:

- Usar la arquitectura **MVVM** (lógica de negocio en ViewModels, acceso a datos en Servicios, estructura en Modelos, presentación en Vistas).
- Aplicar las **9 Reglas de Código Limpio** auditadas: DRY, Fail Fast, sin números mágicos, buenos nombres, sin estado global mutable, no usar `print()`, formato con llaves, un propósito por variable, y comentarios útiles con DartDoc.
- Seguir las convenciones de nomenclatura Dart: `camelCase` para variables y métodos, `PascalCase` para clases, `SCREAMING_SNAKE_CASE` para constantes.
- Ejecutar `dart format .` antes de hacer commit.
- Agregar o actualizar las pruebas unitarias en `test/` para la lógica modificada.

7.3 Code Review Obligatorio

El code review es obligatorio para todas las modificaciones. Otro desarrollador revisa el código buscando:

- Errores lógicos en el flujo de datos reactivo (`Stream`, `Provider`, `ChangeNotifier`).
- Violaciones a las reglas de código limpio auditadas.
- Problemas de rendimiento en consultas Firestore o inferencias TFLite.
- Incompatibilidades con el resto de los módulos del sistema.

Ejemplo específico: si se corrige un bug en `NotificacionServicio._buildNotifData()`, el desarrollador modifica `lib/servicios/notificacion_servicio.dart`, actualiza cualquier prueba unitaria relacionada en `test/`, documenta el cambio en comentarios DartDoc, y crea un pull request con descripción detallada del problema y la solución aplicada.

7.4 Documentación de la Modificación

La documentación de modificaciones incluye:

- **Actualización de comentarios DartDoc:** Si se modifica un método público, se actualizan los comentarios `///` para reflejar los cambios en los parámetros, retorno y comportamiento.
- **Actualización del CHANGELOG.md:** Agregar una entrada con el ID de solicitud, tipo de cambio (Fixed / Added / Changed / Removed) y descripción clara.
- **Actualización de documentación técnica:** Si se modifica la arquitectura (por ejemplo, agregar un nuevo Servicio o ViewModel), actualizar el `README.md` del proyecto.

8. Fase 4: Aceptación y Revisión del Mantenimiento

La fase de aceptación asegura que la modificación cumple con los requisitos antes del despliegue a producción. Esta fase es crítica porque el sistema atiende rescates de mascotas activos en Tacna y cualquier regresión puede afectar a usuarios en situaciones de urgencia.

8.1 Validación por el Área Solicitante

Se prepara un ambiente de staging que replica producción con datos de prueba reales (reportes y avistamientos sintéticos en las coordenadas de Tacna). El proceso incluye:

- Demostración de la funcionalidad modificada al usuario solicitante o al docente evaluador.
- Ejecución de la suite de pruebas unitarias: `flutter test` debe arrojar 100% de casos aprobados.
- Validación manual de los flujos de usuario afectados por la modificación.
- Recopilación de feedback y documentación de observaciones.

8.2 Checklist de Revisión Final Antes del Despliegue

Criterio	Estado
Código revisado y aprobado en code review	☐
Todas las pruebas unitarias pasan (<code>flutter test</code>)	☐
Archivo formateado con <code>dart format</code> .	☐
Documentación DartDoc actualizada	☐
Entrada en CHANGELOG.md agregada	☐
Aceptación del solicitante documentada	☐
Plan de rollback preparado (tag Git de versión anterior)	☐
APK de prueba validado en dispositivo Android físico	☐

9. Fase 5: Migración y Despliegue de la Modificación

La fase de migración comprende la planificación y ejecución del despliegue de la modificación al ambiente de producción del sistema SOS Mascota.

9.1 Planificación de Migración

Para SOS Mascota, se prefieren despliegues en horarios de baja actividad (domingos de 22:00 a 06:00 de la madrugada) para minimizar la interrupción del servicio de rescate activo.

Estrategia	Cuando Aplicar
Big Bang (despliegue completo)	Correcciones críticas P0 que requieren acción inmediata
Gradual / Canary	Cambios grandes en el algoritmo TFLite o modificaciones de esquema en Firestore

El plan de migración documenta:

- Ventana de mantenimiento programada (fecha, hora de inicio y fin estimado).
- Componentes a desplegar (APK actualizado, cambios en `firestore.rules`, nuevos índices en Firestore).
- Orden de despliegue: primero respaldo, luego cambios en Firestore, luego APK.
- Punto de rollback: tag Git con la versión estable anterior.

9.2 Respaldo y Recuperación

El respaldo completo antes de la migración incluye:

- **Export de Firestore:** Export completo de las colecciones `usuarios`, `reportes`, `avistamientos`, `notificaciones`, `chats`, almacenado en Cloud Storage con retención de 30 días.
- **Respaldo de código:** Tag Git con la versión actual antes del cambio (ejemplo: `v1.2.0-pre-mant`).
- **Respaldo de configuración:** Archivos `google-services.json`, `firebase.json`, `firestore.rules` versionados en el repositorio.

El procedimiento de restauración se prueba periódicamente en ambiente de desarrollo para asegurar que funciona correctamente.

9.3 Smoke Tests Post-Despliegue

Después del despliegue, se ejecutan las siguientes verificaciones mínimas:

1. Login con cuenta de voluntario existente en Tacna.
2. Registro de avistamiento con fotografía real y coordenadas GPS.
3. Verificación de que el algoritmo de similitud coseno ejecuta sin errores de inferencia TFLite.
4. Confirmación de que las notificaciones push llegan al dispositivo de prueba.
5. Generación de un afiche PDF de muestra.

10. Fase 6: Retiro de Componentes Obsoletos

La fase de retiro se aplica cuando componentes del sistema SOS Mascota se vuelven obsoletos y deben ser eliminados o reemplazados.

10.1 Componentes Ya Retirados (Laboratorio 01)

En el marco de la auditoría de código del Laboratorio 01, se identificaron y retiraron los siguientes componentes obsoletos:

Componente Retirado	Razón del Retiro	Reemplazo
lib/vista/auth/pantalla_registro_copy.dart	Archivo duplicado sin uso, genera confusión en el equipo	lib/vista/auth/pantalla_registro.dart
lib/vistamodelo/auth/registro_vm_copy.dart	ViewModel duplicado con código idéntico	lib/vistamodelo/auth/registro_vm.dart
Método subirFoto() duplicado en reporte_vm.dart y avistamiento_vm.dart	Viola el principio DRY	ImagenUtil.validarYSubirFoto() centralizado
Llamadas directas a print() en 6 archivos	Viola la regla "No usar print()"	debugPrint() de package:flutter/foundation.dart

10.2 Proceso de Retiro Futuro

Para retiros de componentes futuros, el proceso documenta:

- Componente a retirar (archivos, colecciones Firestore, endpoints).
- Razón del retiro (obsoleto, reemplazado, sin uso).
- Impacto en usuarios activos.
- Sistema de reemplazo.
- Período de transición (mínimo 30 días para funcionalidades con usuarios activos).

Antes de retirar cualquier componente, se archivan los datos históricos relacionados. Los datos de reportes y avistamientos se exportan a formato JSON antes de cualquier migración de esquema en Firestore.

11. Métricas y Monitoreo del Mantenimiento

Para evaluar la efectividad del proceso de mantenimiento del sistema SOS Mascota, se monitorean las siguientes métricas clave:

Métrica	Descripción	Objetivo
Tiempo Medio de Resolución (MTTR)	Tiempo promedio desde recepción hasta resolución de solicitud	< 48 h para P0, < 5 días para P1
Tasa de Éxito de Despliegues	Porcentaje de despliegues sin necesidad de rollback	> 95%
Cobertura de Pruebas Unitarias	Porcentaje de líneas de código cubiertas por flutter test	> 70% (objetivo iteración siguiente)
Número de Errores en Producción	Cantidad de bugs reportados después del despliegue	< 2 por mes
Satisfacción del Usuario	Calificación de voluntarios y dueños de mascotas sobre el sistema	> 4/5
Regresiones Post-Modificación	Casos de prueba que fallan tras aplicar una modificación	0 regresiones

Estas métricas se revisan mensualmente en reunión del equipo para identificar áreas de mejora en el proceso de mantenimiento.

Fuente: Elaboración propia del equipo de trabajo.

12. Roles y Responsabilidades

Los roles involucrados en el mantenimiento del sistema SOS Mascota tienen las siguientes responsabilidades específicas:

Rol	Responsabilidades Principales
Product Owner	Priorización de solicitudes de mantenimiento, aprobación de cambios, comunicación con stakeholders (docente, comunidad de Tacna)
Líder Técnico	Arquitectura, diseño técnico, code review, estimaciones de esfuerzo, aprobación técnica de cambios
Desarrollador Full-Stack	Implementación de cambios en ViewModels, Servicios, Vistas, pruebas unitarias y documentación técnica
AI / Mobile Engineer	Modificaciones en ServicioTFLite, algoritmo de similitud coseno, gestión de modelos .tflite
QA Engineer	Pruebas de integración, validación de flujos de usuario, reportes de calidad, actualización del catálogo de pruebas
Release Manager	Planificación de despliegues, coordinación del cronograma, gestión de versiones y tags Git

13. Referencias

- ISO/IEC 14764:2006 — *Software Engineering — Software Life Cycle Processes — Maintenance*.
- ISO/IEC 12207:2017 — *Systems and software engineering — Software life cycle processes*.
- IEEE Std 1219-1998 — *Standard for Software Maintenance*.
- Documentación oficial de Flutter: <https://docs.flutter.dev>
- Documentación oficial de Firebase: <https://firebase.google.com/docs>
- Repositorio del proyecto: https://github.com/David-Anampa/CONSTRUCCION_SOFTWARE_II_ANAMPA_CHITE